

사출 공정 데이터 기반 불량 예측 대시보드

공정 데이터를 이해 가능한 지표로 바꾸고, 두 분류 모델을 비교해
사용자가 조건을 입력하면 불량 가능성을 확인하는 웹 서비스로 구현

GitHub

https://github.com/00-Dohyun-00/hyundai_autoever_mobility_sw_school_python_project

멤버 및 역할

4인 2파트 구성으로 분석과 구현을 병행

분석 파트

연창 · 우린

- 데이터셋 선정
- 모델링
- 보고서 작성

구현 파트

도현 · 상원

- UI 구현
- 크롤링 구현
- 깃허브 관리

분석 파트가 학습·저장한 모델을 구현 파트가 웹 화면으로 연결하는 구조로 협업했습니다.

분석 과정

1

데이터
수집

CSV
3,500건 · 28컬럼

2

데이터
전처리

스케일링 · 인코딩

3

데이터 학습
(분석)

—
LogisticRegression
— RandomForest

4

검증

학습 80%
검증 20%

5

응용

Flask 대시보드
불량 예측 화면



모델 저장

학습된 모델을 joblib으로 저장



모델 활용

저장된 모델을 예측 화면에서 재사용

STEP 1

데이터 수집

사출 공정 3,500건 · 28개 컬럼 · 목표 변수 defect_label

3,500

전체 샘플

2,809

정상 (80.3%)

691

불량 (19.7%)

변수 그룹

- 식별 · 범주형 machine_id, mold_id, material_code, shift
- 환경 · 수치 ambient_temp, humidity, resin_moisture_pct
- 온도 · 압력 · 속도 barrel_zone, mold_temp, injection/holding_pressure
- 시간 · 운전 fill, holding, cooling, cycle_time, screw_rpm
- 설비 · 캐비티 cushion, clamping_force, cavity_pressure, torque

사출 불량 예측 분석

1 데이터 개요

정상/불량 비율이 불균형한 데이터입니다.

전체 샘플 수

3,500

정상 (0)

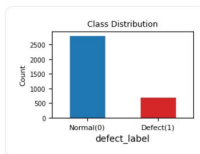
2,809

80.3%

불량 (1)

691

19.7%



▶ 데이터 미리보기 (상위 10행)

불균형 데이터

불량이 약 20%뿐이라 Accuracy는 과대평가됩니다.
Recall · Precision · F1 · AUC를 함께 봐야 합니다.

STEP 2

데이터 전처리

전처리와 분류기를 하나의 Pipeline으로 결합

01

데이터 정리

timestamp 제거
종속, 독립 변수 분리

02

변수 구분

문자열 · 수치 컬럼 구분

03

수치형 처리

데이터 표준화

04

범주형 처리

OneHotEncoder
handle_unknown='ignore'

불균형 대응

Logistic Regression

`class_weight = "balanced"`

Random Forest

`class_weight = "balanced_subsample"`

STEP 2

데이터 전처리

원본 28개 컬럼을 타입별로 나눠 변환하고, 전처리와 분류기를 하나의 Pipeline으로 결합

원본 28컬럼

CSV 그대로

timestamp 제거

식별용 · 학습 불필요

X / y 분리

y = defect_label

타입 자동 구분

문자열 4 · 수치형 22

최종 35개 특성

수치 22 + 원핫 13

문자열 4개

object 타입 → 순서 의미 없음

machine_id	IMM-01 ~ IMM-04	4
mold_id	MD-A12 · MD-B07 · MD-C03	3
material_code	ABS · PA6-GF30 · PP-GF20	3
shift	DAY · EVENING · NIGHT	3

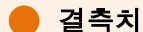
OneHotEncoder(handle_unknown='ignore') → 13개 컬럼

수치형 22개

float64 타입 → 크기 비교 가능

● 환경 · 수지	3	ambient_temp · humidity · resin_moisture
● 배럴 · 금형 온도	4	barrel_zone1~3 · mold_temp
● 압력 · 속도	5	injection · holding · back · speed · cushion
● 시간	4	fill · holding · cooling · cycle
● 설비 · 캐비티	6	screw_rpm/torque · clamping · cavity · oil_temp

형체력 680~960 vs 함수율 0.02~0.32 → StandardScaler 필수



결측치

3개 컬럼 143건 (1.4%)

슬도 49 · 함수율 50 · 캐비티압 44 → 평균 대체



스케일링

StandardScaler

평균 0 · 표준편차 1로 변환해 단위 차이 제거



불균형

class_weight

LR: balanced / RF: balanced_subsample

STEP 3

데이터 학습 (분석)

싱글 모델과 앙상블 모델을 각각 학습해 비교

싱글 Single

Logistic Regression

선형 결정 경계를 학습하는 단일 모델

```
max_iter = 1000
```

```
class_weight = "balanced"
```

→ 계수로 변수 영향 방향을 해석 가능

앙상블 Ensemble

Random Forest

다수의 결정 트리를 학습해 투표로 결정

```
n_estimators = 300
```

```
class_weight = "balanced_subsample"
```

→ 비선형 관계와 변수 상호작용 포착

모델 저장

학습된 두 모델과 예측용 기본값을 joblib으로 저장 → 예측 화면에서 재학습 없이 재사용

```
models/*.joblib
```

검증

학습 데이터 80% (2,800건)

2 분류모델

분류모델 1 - Logistic Regression

분류모델 1 - Logistic Regression

분류모델 2 - Random Forest

ACCURACY

0.609

PRECISION

0.262

RECALL

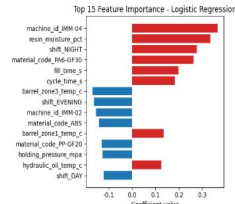
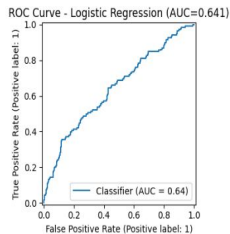
0.543

F1 SCORE

0.354

AUC

0.641



모델	threshold	Accurac y	Precisio n	Recall	F1	AUC
Logistic Regression	0.50	0.609	0.262	0.544	0.354	0.641
Random Forest	0.30	0.774	0.375	0.217	0.275	0.658

Random Forest : Accuracy · AUC 우세

Logistic Regression : Recall 우세 (0.544)

Recall(실제 불량 중에서 모델이 잡아낸 비율)에서 큰 차이

STEP 4.5

하이퍼파라미터 튜닝

GridSearchCV · 5-Fold 교차검증 · scoring = f1

Logistic Regression

규제 강도 (regularization)

작을수록 규제가 강해져 모델이 단순해지고 과적합을 막습니다.

선형 모델의 성능을 가장 직접적으로 좌우하는 파라미터입니다.

후보 [0.01, 0.1, 1, 10, 100] 로그 스케일 탐색

Random Forest

n_estimators

트리 개수 — 일정 수 이상은 시간만 증가

max_depth

트리 깊이 — 깊으면 과적합, 얇으면 패턴 놓침

n_estimators [100, 200, 300, 500]

max_depth [5, 10, 20, None]

평가 기준을 f1 으로 잡은 이유

● Accuracy

전부 정상으로 찍어도 80%
불균형에 무력

● Recall

전부 불량으로 찍으면 1.0
혼자 보면 함정

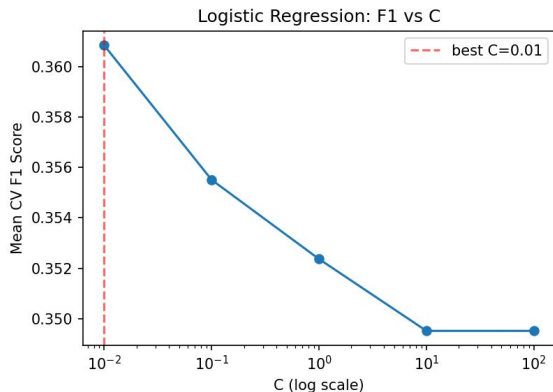
● F1

Precision · Recall의 조화평균
균형점을 찾는 기준

STEP 4.5

파라미터 변화에 따른 F1

교차검증 평균 F1 기준 · 최적 조합 : $C = 0.01$ / $\text{max_depth}, \text{n_estimators} = 5, 300$

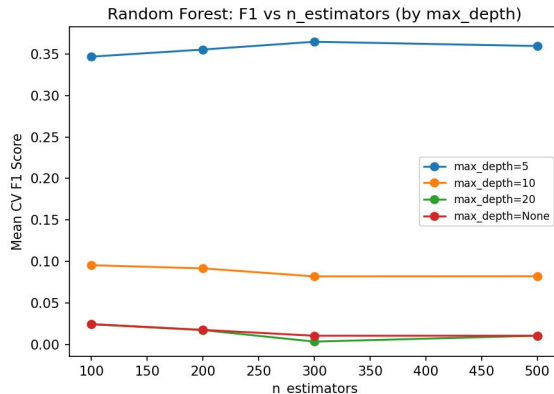


Logistic Regression : F1 vs C

C가 커질수록 F1이 완만하게 하락합니다.

다만 변동 폭이 0.349 ~ 0.361로 매우 좁아,

튜닝 효과 자체는 크지 않습니다.



Random Forest : F1 vs n_estimators

n_estimators는 거의 영향이 없는 반면

max_depth에 따라 F1이 0.365 → 0.01로

급락합니다.

왜 급락했나

트리가 깊어질수록 예측 확률이 낮은 쪽으로 몰려

판정 기준 0.5를 넘지 못합니다.

depth = 5 171건
최대 0.662

depth = 10 11건
최대 0.573

depth = 20 0건
최대 0.500

depth = None 1건
최대 0.507

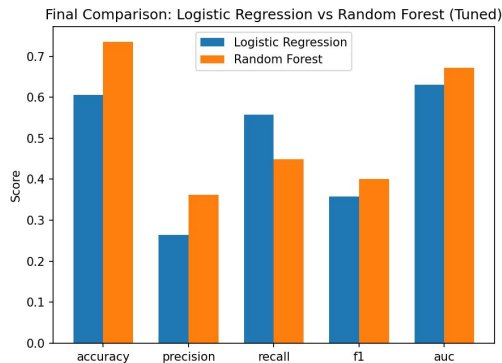
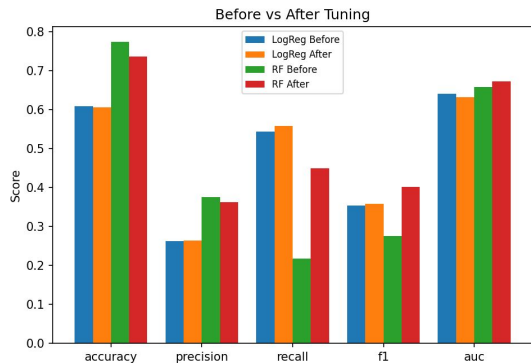
→ 0.5 이상으로 판정된 건수

깊은 트리는 사실상 전부 정상 판정

STEP 4.5

튜닝 전후 비교

테스트셋 700건 기준



튜닝 효과

Random Forest

F1 0.275 → 0.401

Recall 0.217 → 0.449

Logistic Regression

F1 0.354 → 0.358

	Accuracy	Precision	Recall	F1	AUC
LogReg 전	0.609	0.262	0.543	0.354	0.641
LogReg 후	0.606	0.264	0.558	0.358	0.631
RF 전	0.774	0.375	0.217	0.275	0.658
RF 후(th=0.5)	0.736	0.363	0.449	0.401	0.673
RF 후(th=0.3)	0.197	0.197	1.000	0.329	0.673

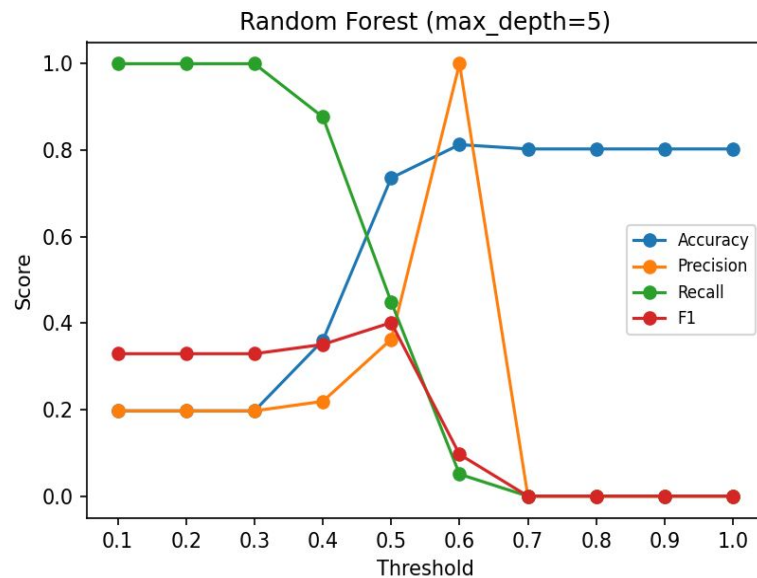
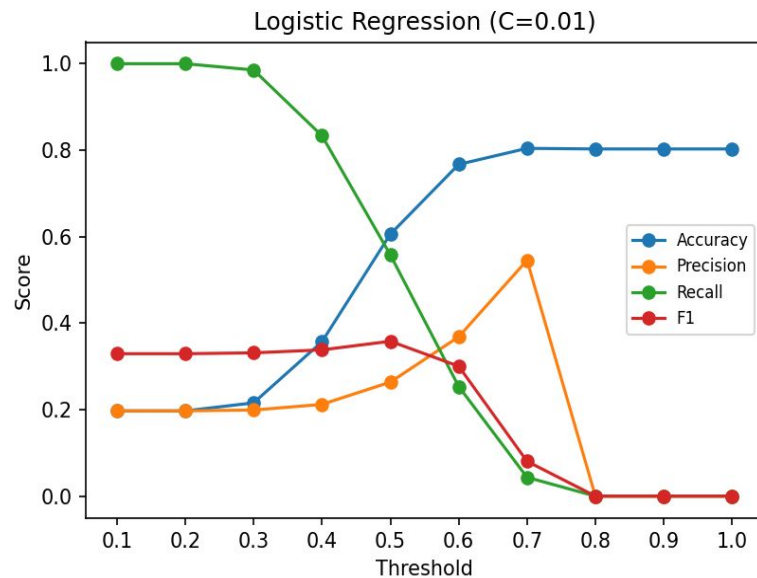
최종 선택

튜닝 후 Random Forest가 F1 0.401, AUC 0.673으로 앞섬.

다만 Recall은 Logistic Regression이 0.558로 여전히 높은 편이므로 고려해야함

추가 실험

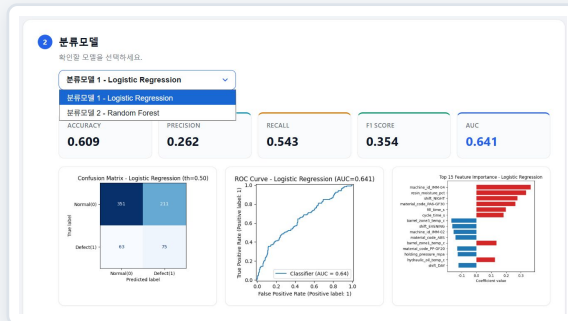
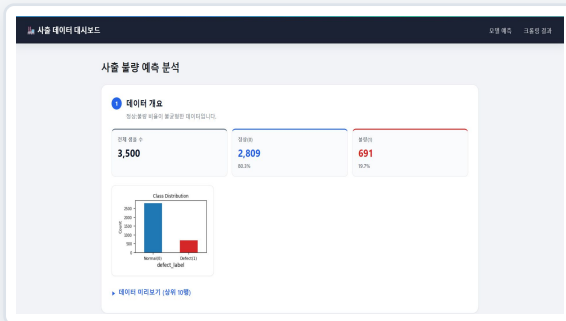
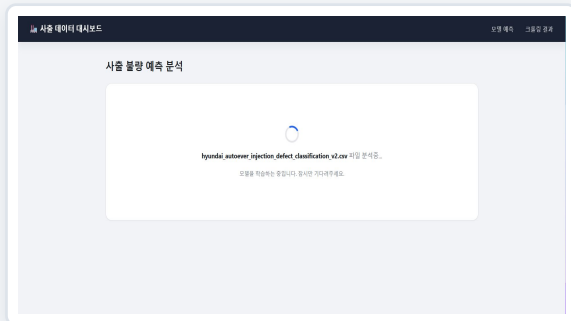
Metrics vs Threshold (0.1~1.0), Tuned Models



STEP 5

응용 : 웹 대시보드

데이터 현황 → 모델 성능 → 해석의 3단 구성



1 분석 실행

CSV 로드와 모델 학습 진행 표시

2 데이터 현황

샘플 수 · 클래스 분포 · 상위 10행

3 모델 성능

지표 · 혼동행렬 · ROC · 중요도

구현 포인트

로딩 화면을 먼저 띄우고, JS fetch로 받은 분석 결과 **fragment**를 교체합니다.

차트는 **matplotlib base64 PNG** · **React·Vue 없이** 서버 렌더링 + **vanilla JS**

STEP 5

응용 : 파라미터 입력으로 시뮬레이션 실행

저장된 분석 모델을 불러와 사용자가 원하는 파라미터로 시뮬레이션으로 직접 확인

활용 설정

모델 선택

Logistic Regression

LogisticRegression

max_iter 1000

class_weight balanced

수지 수분율 (%)

resin_moisture_pct

금형 온도 (°C)

mold_temp_c

사출 압력 (MPa)

injection_pressure

보압 (MPa)

holding_pressure

사출 속도 (MM/S)

injection_speed_n

충전 시간 (S)

fill_time_s

냉각 시간 (S)

스크류 회전수 (RPM)

1

모델 선택

Logistic Regression 또는 Random Forest

2

공정 조건 10개 입력

수분율 · 온도 · 압력 · 속도 · 시간 · 형체력

3

확률 계산

저장된 모델의 predict_proba() 호출

4

정상 / 불량 판정

모델별 threshold 기준으로 판정 후 표시

사용 모델 Logistic Regression

resin_moisture_pct 1

mold_temp_c 1

injection_pressure_mpa 1

holding_pressure_mpa 1

injection_speed_mm_s 1

fill_time_s 1

cooling_time_s 1

screw_rpm 1

back_pressure_mpa 1

clamping_force_kn 1

불량 확률 100.0%

예측 결과 (defect_label) 불량 (1)

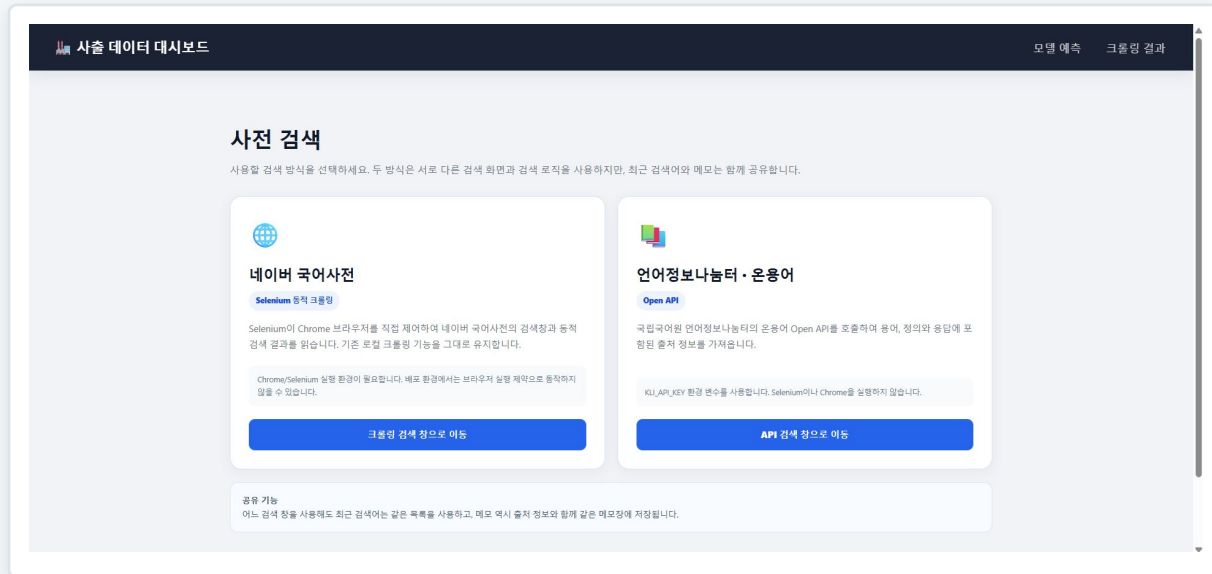
입력 폼에 없는 나머지 특성은 학습 데이터의 수치형 평균과 범주형 최빈값으로 자동 보완됩니다.

분석 모델 시연 영상

STEP 5

응용 : 크롤링

Selenium 크롤링과 Open API를 선택적으로 사용해 단어의 뜻과 출처를 확인하는 사전 검색 기능



원하는 검색 방식 선택

네이버 국어 사전 크롤링과

언어정보나눔터 API를 통한

두가지 검색 방식 중 원하는 것을 선택

네이버 국어사전 - Selenium 동적 크롤링

- 실제 웹 페이지 검색
- Chrome Driver 실행
- 검색 결과의 뜻 자동 추출

언어정보나눔터 - Open API

- API를 통한 검색
- JSON 결과 파싱
- 용어 · 뜻 · 출처 확인

이유 : 크롤링

→ 검색 방식 선택

로봇이 API 검색 필요로 함

네이버 국어사전 검색

Selenium이 실제 Chrome 브라우저를 제어하여 네이버 국어사전의 동적 화면에서 검색 결과를 가져옵니다.

네이버 국어사전 검색

Selenium 동적 크롤링 방식으로 검색합니다. 검색 결과의 예외에는 현재 데이터 출처가 함께 표시됩니다.

현재 검색 방식

현재 출처 - 네이버 국어사전

Selenium 동적 크롤링

Selenium이 실제 Chrome 브라우저를 제어하여 네이버 국어사전의 동적 화면에서 검색 결과를 가져옵니다.

검색할 단어를 입력하세요

검색

최근 검색어 - 두 검색 방식 공통

최근 검색어가 없습니다.

현재 검색된 네이버 국어사전에서 검색할 단어를 입력해주세요.

단어 메모장

직접도 작성

최근 요거가

네이버의 온유이 결과를 입력 메모장 수 있으며, 각 메모에는 실제 출처가 저장됩니다.

저장된 메모가 없습니다.

[▶ 현재 방식 안내](#)
[새버리 호출형 검색 방식으로 이동](#)

국립국어원 언어정보나눔터 - 온용어 검색

국립국어원 언어정보나눔터의 온용어 Open API를 호출하여 용어와 정의를 가져올입니다.

국립국어원 언어정보나눔터 - 온용어 검색
 Open API 방식으로 검색합니다. 검색 결과와 예외되는 한때 데이터 출처가 함께 표시됩니다.

현재 검색 방식

현재 출처 - 국립국어원 언어정보나눔터 - 온용어 [Open API](#)

국립국어원 언어정보나눔터의 온용어 Open API를 호출하여 용어와 정의를 가져옵니다.

검색

최근 검색어 : 두 점체 방식 공통

최근 검색 기록과 같습니다.

현재 신청된 국립국어원 언어정보나눔터 - 온용어에서 검색한 단어를 입력해주세요.

단어 메모장

데이터바가 온용어 결과를 함께 메모할 수 있으며, 가 메모에는 실제 출처가 저장됩니다.

작성도 저장
바로 보기

저장한 메모가 없습니다.

- 실제 웹 페이지 검색
- Chrome Driver 실행
- 검색 결과의 뜻 자동 추출

- API를 통한 검색
- JSON 결과 파싱
- 용어 · 뜻 · 출처 확인

STEP 5

응용 : 크롤링

Selenium으로 네이버 국어사전을 검색해 뜻을 추출하고, 결과를 메모로 정리

네이버 국어사전 검색

검색어를 입력하면 네이버 국어사전의 뜻을 가져옵니다.

● 검색 출처 : 네이버 국어사전

수분율

검색

최근 검색어

전체 삭제

수분율 x

"수분율" 검색 결과

총 17개의 결과를 찾았습니다.

네이버 국어사전

원문에서 확인 >

수분율 뜻 1

명사 자연 일반 시료의 임의 상태인 원중량 무게와 건조 후 무게의 차를 건조 후 무게로 나눈 값을 백분율로 변환한 것.

네이버 국어사전

메모에 추가

사전 검색

네이버 국어사전 검색

국어를 입력하면 네이버 국어사전의 뜻을 가져옵니다.

● 검색 출처 : 네이버 국어사전

수분율

검색

최근 검색어

전체 삭제

수분율 x

원문에서 확인 >

수분율 뜻 1

명사 자연 일반 시료의 임의 상태인 원중량 무게와 건조 후 무게의 차를 건조 후 무게로 나눈 값을 백분율로 변환한 것.

네이버 국어사전

메모에 추가

1 사전 검색 화면

- 검색창
- 검색 결과 영역
- 단어 메모장

단어 메모장

원문에서 단어를 메모한 뒤 TXT 파일로 저장할 수 있습니다.

파일로 저장

메모 초기화

공백

명사 공백으로 만든 가루집.

출처 : 네이버 국어사전

수분율

2020. 8. 27. 오후 2:54:21

화학 각 섬유에 함유된 수분율 일정하게 규정된 비율. 온도 25℃, 습도 65% 상태에서 흡수하는 수분율을 표준으로 한다.

출처 : 네이버 국어사전

2 검색 결과

뜻 하나를 카드 하나로 표시 · 원문 링크와 「메모에 추가」 버튼 제공

3 단어 메모장

단어 · 뜻 · 출처 · 시각 저장 → TXT 파일로 내려받기

검색 방식은 분리되어 있지만 최근 검색어와 메모는 하나의 사용자 데이터로 공유

STEP 5

응용 : API

API를 통해 언어정보나눔터를 검색해 뜻을 추출하고, 결과를 메모로 정리

국립국어원 언어정보나눔터 · 온용어 검색

Open API 방식으로 검색합니다. 검색 결과와 메모에는 현재 데이터 출처가 함께 표시됩니다.

현재 검색 방식

현재 출처 · 국립국어원 언어정보나눔터 · 온용어 [Open API](#)

국립국어원 언어정보나눔터의 온용어 Open API를 호출하여 용어와 정의를 가져옵니다.

수분율

검색

최근 검색어 · 두 검색 방식 공유

수분율 × 언명 × 사기 × 세상에 존재하지 않는단어 × ㅋㅋㅋ ×

전체 삭제

“수분율” 검색 결과

총 5개의 뜻을 찾았습니다.

출처 · 국립국어원 언어정보나눔터 · 온용어 [방식 · Open API](#) 출처 사이트에서 확인 ↗

수분율 뜻 1

시료의 염의 상태의 무게(원중량 : W)와 건조후의 무게(W')와의 차를 건조후 무게로 나누어 얻은 백분율.

데이터 출처 · 농촌진흥청 용어집 · 농림 용어 사전

[메모에 추가](#)

국립국어원 언어정보나눔터 · 온용어 검색

Open API 방식으로 검색합니다. 검색 결과와 메모에는 현재 데이터 출처가 함께 표시됩니다.

현재 검색 방식

현재 출처 · 국립국어원 언어정보나눔터 · 온용어 [Open API](#)

국립국어원 언어정보나눔터의 온용어 Open API를 호출하여 용어와 정의를 가져옵니다.

검색어 입력

검색

최근 검색어 · 두 검색 방식 공유

전체 삭제

원인 언어의 사용처를 알려드립니다. 온용어에서 입력한 언어를 입력해주세요.

단어 메모장

해당 단어의 온용어 결과를 함께 메모할 수 있으며, 각 메모에는 실제 출처가 저장됩니다.

새 메모 작성

전체 삭제

단어 메모장
단어와 뜻을 함께 메모할 수 있으며, 각 메모에는 실제 출처가 저장됩니다.
단어: 수분율
뜻: 시료의 염의 상태의 무게(원중량 : W)와 건조후의 무게(W')와의 차를 건조후 무게로 나누어 얻은 백분율.
출처: 농촌진흥청 · 농림 용어 사전
저장 일시: 2023. 6. 28. 오전 11:58:48

1 사전 검색 화면

- 검색창
- 검색 결과 영역
- 단어 메모장

2 검색 결과

뜻 하나를 카드 하나로 표시 · 원문 링크와 「메모에 추가」 버튼 제공

3 단어 메모장

단어 · 뜻 · 출처 · 시각 저장 → TXT 파일로 내려받기

검색 방식은 분리되어 있지만 최근 검색어와 메모는 하나의 사용자 데이터로 공유

STEP 5

크롤링 구현 상세

동작 흐름 · 주요 기능 · 오류 처리

크롤링 흐름

- 1 검색어 입력
로딩 화면 · 버튼 비활성화
- 2 Chrome Driver 실행
Headless 모드
- 3 네이버 국어사전 접속
검색창 탐색 후 ENTER
- 4 검색 결과 대기
WebDriverWait
- 5 뜻 추출
p.mean 수집 · 중복 제거
- 6 결과 반환 · 화면 출력
Flask → Jinja2

API 흐름

- 1 검색어 입력
로딩 화면 · 버튼 비활성화
- 2 API Key 확인
환경변수 인증키 확인
- 3 API 요청
검색어 · Key 전달
- 4 JSON 응답
검색 결과 데이터 수신
- 5 뜻 · 출처 파싱
용어 · 정의 · 출처 추출
- 6 결과 반환 · 화면 출력
Flask → Jinja2

공용 기능

- 뜻별 결과 카드
원문 링크 함께 제공
- 검색어 ↔ 결과 단어 비교
다르면 경고 표시 · 결과는 유지
- 단어 더보기 자동 클릭
실패 시 JavaScript 클릭 재시도
- 최근 검색어 5개
클릭 시 즉시 재검색
- 메모 추가
중복 방지 · 전체 추가 · 개별 삭제
- 메모 TXT 저장
날짜 포함 파일명으로 내려받기

사용 기술

Selenium

Chrome Driver

Open API

Jinja2

sessionStorage

부분 실패 시
결과 유지 + 경고

검색 기술은 분리하고 사용자 경험은 하나로
유지

STEP 5

사전 검색 오류 처리

검색 방식별 오류를 구분하고, 사용자 안내와 개발자 로그를 분리해 처리

17

가지 오류 코드로 분류해

원인별 안내 화면 표시

[Selenium 오류]

- 1 **DRIVER_START_FAILED**
Chrome Driver 실행 실패
- 2 **SITE_CONNECTION_FAILED**
네이버 국어사전 접속 실패
- 3 **RESULT_LOAD_TIMEOUT**
검색 결과 로딩 시간 초과
- 4 **ACCESS_BLOCKED**
자동화 접근 제한
- 5 **NO_RESULTS**
검색 결과 없음

[Open API 오류]

- 1 **API Key 오류**
환경변수 인증키 누락 · 인증 실패
- 2 **API 요청 실패**
HTTP · 네트워크 요청 오류
- 3 **응답 처리 오류**
JSON 응답 또는 결과 파싱 실패
- 4 **NO_RESULTS**
검색 결과 없음

[공통 오류 처리]

- **사용자 화면**
코드 · 제목 · 설명 · 확인 방법
- **서버 로그**
상세 예외 내용 기록
- **부분 실패**
검색 결과 확보 시
결과 유지 + 경고 표시

크롤링 시연 영상

STEP 5

프로젝트 구조와 동작 흐름

서버 렌더링 + 최소한의 vanilla JS (React · Vue 미사용)

파일 구조

<code>app.py</code>	Flask 진입점 · 라우트 정의
<code>data/</code>	원본 CSV (28컬럼 · 3,500행)
<code>analysis/</code>	
<code>analyzer.py</code>	분석 · 예측 진입점
<code>defect_classification.py</code>	학습 · 지표 · 차트 · 저장
<code>crawler/crawler.py</code>	Selenium 네이버 사전 크롤링 언어정보나눔터 API 요청
<code>templates/</code>	base · csv_analysis · model 등
<code>static/css/style.css</code>	공통 스타일시트
<code>models/</code>	학습 모델 (git 추적 제외)

동작 흐름

홈 /

- 1 로딩 화면 렌더링
- 2 JS fetch → /analysis-content
- 3 모델 학습 → joblib 저장
- 4 차트 base64 → fragment 반환

예측 /model

- 1 joblib.load 로 모델 로드
- 2 입력 10개 + 나머지 자동 보완
- 3 정상 / 불량 + 확률 표시

시연 시 주의

models/ 는 git 추적에서 제외되어 있어 서버를 새로 띄운 직후에는 비어있음

반드시 홈 화면(/)의 분석을 먼저 완료해야 /model 예측이 동작합니다.

학습 전 예측 시도 시 오류 없이 "예측 결과가 없습니다."로 표시됩니다.

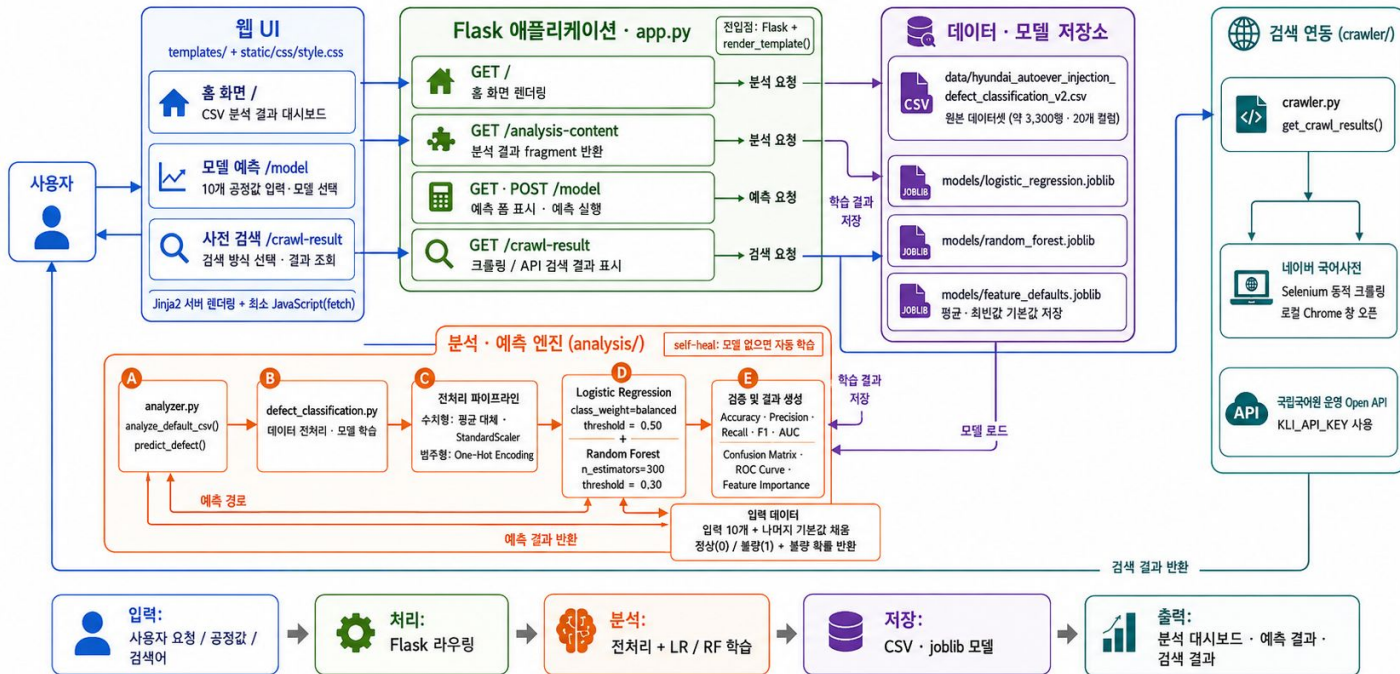
STEP 5

프로젝트 구조와 동작 흐름

서버 렌더링 + 최소한의 vanilla JS (React · Vue 미사용)

사출 데이터 대시보드 전체 시스템 아키텍처

Flask + Jinja2 + Machine Learning + 크롤링 / Open API

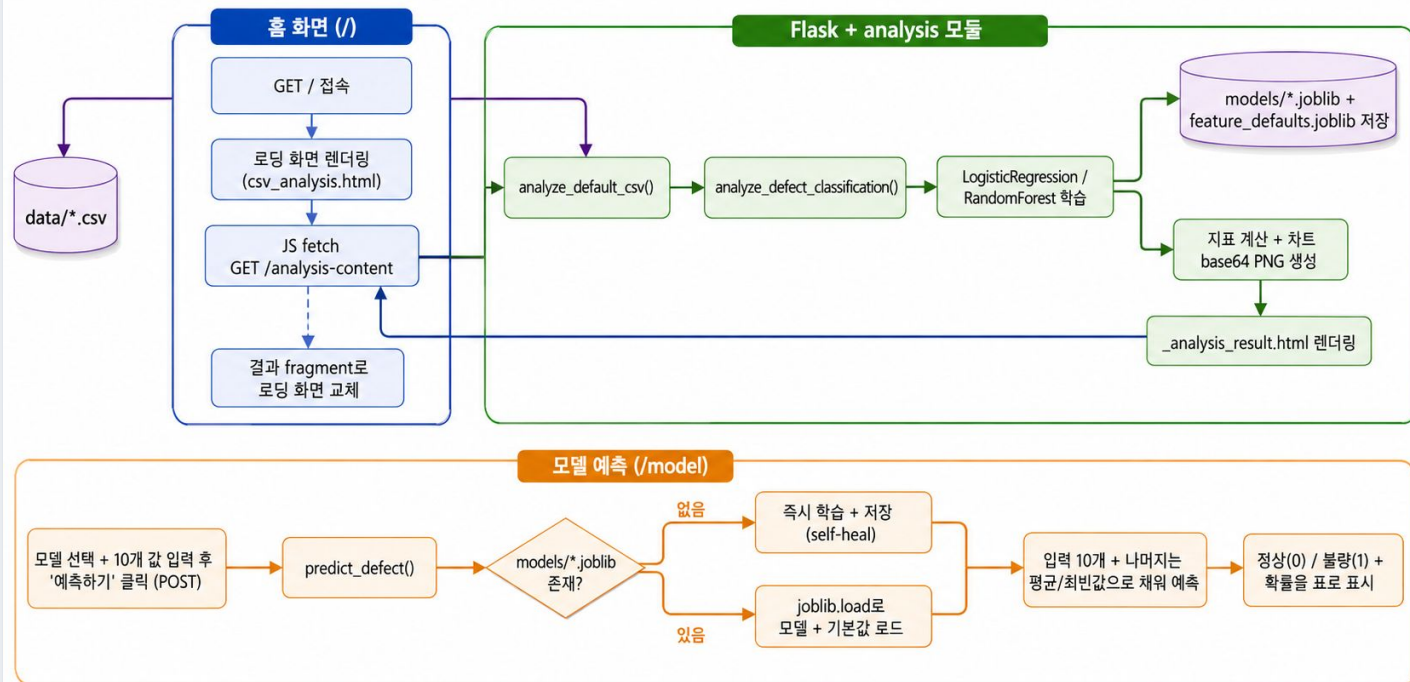


STEP 5

프로젝트 구조와 동작 흐름

서버 렌더링 + 최소한의 vanilla JS (React · Vue 미사용)

분석 화면 · 예측 흐름 아키텍처



분석 결과와 인사이트

3 결론

Logistic Regression: AUC 0.641, Recall 0.543 (threshold=0.50)

Random Forest: AUC 0.658, Recall 0.217 (threshold=0.30)

두 모델의 AUC 차이가 크지 않습니다 (약 0.017 차이). 모델을 더 복잡한 것으로 바뀌어도 성능이 극적으로 개선되지 않는 것으로 보아, 현재 성능의 병목은 모델 종류보다 불량(1) 데이터 수 자체가 적은 클래스 불균형에 있는 것으로 판단됩니다. 두 모델 모두 threshold(판정 기준)를 낮출수록 recall이 오르고 precision이 떨어지는 트레이드오프를 보이며, 이는 AUC로 대표되는 모델의 잠재력과 실제 판정 결과 (recall/precision)가 서로 다른 것임을 보여줍니다.

Feature Importance 상 resin_moisture_pct(수지 수분율), injection_pressure_mpa(사출 압력) 등이 두 모델 모두에서 상위권으로 나타났습니다.

다음 개선 단계

- 공정 조건 관리 강화
- 불량 데이터 추가 확보
- 교차검증 · 시간순 검증

● 모델 튜닝 및 선정

제조 데이터이므로 Recall을 고려한 모델 선정 필요

● 주요 과제는 모델 종류가 아님

불량 클래스의 데이터 부족과 불균형이 더 큰 제약일 가능성

● 상위 영향 변수 확인

resin_moisture_pct (수지 수분율), injection_pressure_mpa (사출 압력)

한계와 개선 방향

구현된 기능과 향후 계획의 구분

현재 한계

- 네이버 HTML 구조와 Chrome 실행 환경,
외부 API 서비스 및 인증키에 의존
- 단일 train/test split — 교차검증과 시간순 검증 필요
- Feature Importance는 인과관계가 아니므로 현장 검증 필요
- 실제 운영에서는 오탐 · 미탐 비용 반영 필요

개선 방향

- 불량 샘플 추가 수집
성능 병목 해소의 출발점
- 교차검증 필요
단일 분할의 우연성 제거
- SHAP 기반 설명가능성
판정 근거를 현장에 제시
- 검색 기록 · 메모 DB 저장
sessionStorage → 영구 보관

결론



수집 · 전처리

3,500건의 사출 공정 데이터를
전처리와 함께 분석



학습 · 검증

싱글과 앙상블 모델을
비교 검증



응용

Flask 웹 대시보드와
신규 조건 예측 기능으로 연결

데이터를 읽는 분석에서 공정 의사결정을 돕는 서비스로 확장한 프로토타입입니다.

다음 단계는 더 많은 불량 데이터와 현장 비용을 반영해 예측 신뢰도와 운영성을 높이는 것입니다.